

Q. Create a dataframe using dictionary which includes student name, semester, marks, height, Age, study hours. Also include some null value

- (i) value count of student in sems
- (ii) outliers in marks column.
- (iii) correlation in marks & study hours.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data = {  
    "name": ["A", "B", "C", "D", "E"],  
    "semester": [2, 2, 1, 1, 1],  
    "Marks": [100, 100, 99, 97, 95],  
    "age": [18, 17, 18, np.nan, np.nan],  
    "height": [162, 165, 177, 186, 170],  
    "Study hours": [8, 9, 4, np.nan, np.nan]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
print(df["semester"].value_counts())
```

```
q1 = np.percentile([90, 100], 25)
```

```
q3 = np.percentile([90, 100], 75)
```

```
iqr = q3 - q1
```

```
lower = [(-1.5) * iqr] + q1
```

```
upper = [(1.5) * iqr] + q3
```

```
print(lower)
```

```
print(upper)
```

```
marks_studyhours = df["marks"].corr(df["studyhours"])
```

```
print("correlation is:", marks_studyhours)
```

```
x = [100, 100, 99, 97, 95]
```

```
y = [8, 9, 4, np.nan, np.nan]
```

```
plt.plot(x, y)
```

```
plt.show()
```


Q There are 2 data set given in which:
→ one dataset contains medical records.
→ second dataset contains patient records.

Now preprocess / clean the dataset, then merge them together.

```
import pandas as pd  
df1 = pd.read_csv("C:\\users\\Gru\\Downloads\\medical_records.csv")
```

```
print(df1)
```

```
df1.head(10)
```

```
df1.tail(10)
```

```
df1.info()
```

```
df1.isnull()
```

```
df2 = pd.read_csv("C:\\users\\Gru\\Downloads\\patient_details.csv")
```

```
print(df2)
```

```
df2.head(10)
```

```
df2.tail(10)
```

```
df2.info() →
```

```
df2.isnull()
```

```
df2.duplicated()
```

```
merge_right = pd.merge(df1, df2, how="right")  
merge_right
```

```
merge_left = pd.merge(df1, df2, how="left")  
merge_left
```

```
merge_outer = pd.merge(df1, df2, how="outer")  
merge_outer
```

```
merge_inner = pd.merge(df1, df2, how="inner")  
merge_inner
```

```
print(df1.describe())
```

```
print(df2.describe())
```

```
print(df1.isnull().sum().sum())
```

```
print(df2.isnull().sum().sum())
```

```
print(df1.duplicated())
```

```
print(df2.duplicated())
```

```
hyper_pat = df1[df1['diagnosis'] == "hypertension"]  
print(hyper_pat)
```

```
df2 = df2.replace(  

```


Q3

```
def dfs(start, goal, graph, vis=None):
```

```
    if vis is None:
        vis = set()
```

```
    print(start)
```

```
    if start == goal:
        return True
```

```
    vis.add(start)
```

```
    for v in graph.get(start, []):
```

```
        if v not in vis:
```

```
            if dfs(v, goal, graph, vis):
                return True
```

```
    return False
```

```
graph = {
```

```
    'A': ['B', 'C']
```

```
    'B': ['A', 'C', 'D']
```

```
    'C': ['A', 'B', 'E']
```

```
    'D': ['B', 'E', 'F']
```

```
    'E': ['C', 'D', 'F'],
```

```
    'F': ['D', 'E']
```

```
}
```

```
print(dfs('A', 'F', graph))
```

Q1 what is a graph and why do we use it?
(2em) and why

Q2 Types of graphs

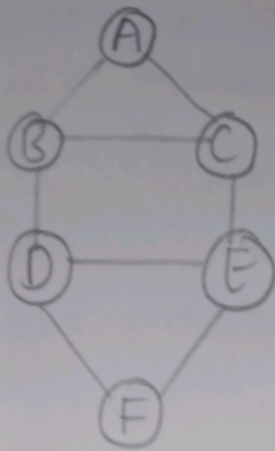
Q3 what is

any 5 real time applications of dfs

Q4 what is BFS? why do we use it?
what are its applications

Q5 state the difference between dfs & BFS

Q why we're doing normalisation and min-max



A - B
 A - C
 B - C
 B - D
 C - E
 D - E
 D - F
 E - F
 F - D
 F - E

Denote:-
 ↳ parent - U
 ↳ child - V

write code for DFS for the given graph.

Ans:- def dfs(start, goal, graph, vis=None):
 if vis is None:
 vis = set()
 print(start)
 if start == goal:
 return True
 vis.add(start)
 for child in graph[start]:
 if child not in vis:
 if dfs(child, goal, graph, vis):
 return True

~~graph = {}~~
 n = int(input("enter no. of nodes = "))
 for i in range(n):
 k = input("enter node: ")
 graph[k] = []
 m = int(input("how many neighbours: "))
 for j in range(m):
 v = input("enter neighbour: ")
 graph[k].append(v)

```

graph
s = input("enter starting point: ")
E = input("enter ending point: ")
print(dfs(s, E, graph))

```

Output

Enter no. of nodes = 6

enter node: A

how many neighbours: 2

enter neighbours: B

enter neighbour: C

enter node: B

how many neighbours: 3

enter neighbours: A

enter neighbours: B

enter neighbour: E

enter node: D

how many neighbours: 3

enter neighbours: B

enter neighbour: E

enter neighbour: F

enter node: E

how many neighbours = 3

enter neighbour: C

enter neighbour: D

enter neighbour: F

enter node: F

how many neighbours: 2

enter neighbour: D

enter neighbour: E

enter starting point: A

enter ending point: F

A

B

C

E

D

F

True

Q4 create a dataset for medical purpose which have columns like patient id, age, name, diagnosis, sugar levels, B.P, income of the patient
normalisation of age, sugar level, income.

Import pandas as pd

Import matplotlib.pyplot as plt

```
data = { "Patient id": ["P01", "P02", "P03", "P04", "P05"],  
        "Name": ["Rahul", "Raj", "Amit", "Neha",  
                 "Arjun"],  
        "Age": [25, 40, 35, 50, 29],  
        "Sugar-level": [110, 159, 139, 170, 115],  
        "B.P": [120, 140, 130, 150, 125],  
        "Income": [3000, 5000, 4000, 6000, 4500]  
    }
```

```
n = ["Age", "sugar-level", "income"]
```

```
df[n] = (df[n] - df[n].min()) / df[n].max() -  
        df[n].min()
```

```
print(df)
```

code for Astar

import network as nn

graph = nn.Graph()

graph

while True:

choice = int(input("Insert nodes and
centre edge! press 1 for
yes & 2 for no"))

if choice == 1:

node1 = input("enter node1").strip().upper()

node2 = input("enter node2").strip().upper()

weight = int(input("enter weight ^{btw node1 & node2}"))

graph.add-edge(node1, node2, weight =
weight)

else:

break

heuristic = {}

nodes = list(graph.nodes())

print("enter heuristic value for each other")

for node in nodes:

heuristic[node] = int(input(f'h{node} = '))

def n(n1, n2):

Return heuristic[n1]

st = input("start node").strip().upper()

go = input("goal node").strip().upper()

path = nn.astar-path(graph, st, heuristic=h,
weight=weight)

print("a* path:", path)

Output:

want to insert & enter edges (press 1 for yes & 2 for no)

Enter node1: a

Enter node2: b

Enter the weight btw node1 & node2: 4

want to insert & enter edges (press 1 for yes & 2 for no)

Enter node1: a

Enter node2: c

Enter the weight btw node1 & node2: 5

want to insert & enter edges (press 1 for yes & 2 for no)

Enter node1: b

Enter node2: c

Enter the weight btw. node1 & node2: 6

want to insert & enter edges (press 1 for yes & 2 for no)

Enter heuristic values for each node:

Enter $h(A)$: 3

Enter $h(B)$: 5

" $h(C)$: 2

Start node: a

Goal node: c

A* Path: [A, C]

① a graph is a non-linear data structure consisting of :

- vertices (nodes) - Represent entities
- Edges (links) - Represent relationships between entities ($G=(V,E)$)

graphs are required to represent real world problems where relationships between objects exists

Reasons:

- ① To represent networks
- ② To solve path finding problems.
- ③ To model dependencies.
- ④ To represent hierarchical and non-hierarchical relationships
- ⑤ To perform searching and transversal operations

Real life

- social media (friends network)
- google maps (shortest path discovery)
- Airline routes
- web pages (links between pages)

② Based on directions.

① undirected graph

→ Edges have no direction

→ if A is connected to B, B is also connected to A

Eg → A — B

② directed graph (Di-graph)

→ Edges ~~are~~ have direction

→ $A \rightarrow B$ does not mean $B \rightarrow A$

③ Based on weight

① weighted graph

→ Edges have weight (distance, cost, time)

Eg → A $\xrightarrow{5}$ B

② unweighted graphs

→ Edges have no weight

④ Based on cycles

① cyclic graph

→ contains at least one cycle

② A cyclic graph

→ No cycles

Eg → tree

③ DFS (Depth first search)

It is a graph transversal algorithm that goes as deep as possible first and then backtracks. It uses stacks or recursion.

use of DFS:

- To explore all the nodes nearby
- To detect cycles
- To check connectivity
- for path finding
- for topological sorting

Real life Applications:-

- ① maze solving
- ② detecting cycles in network
- ③ file system navigation
- ④ Topological sorting
- ⑤ Solving puzzles (Sudoku)

④ BFS (Breadth first search)

It is a transversal algorithm that visits nodes level by level

→ It uses queue (FIFO)

uses of BFS

- To find shortest path in unweighted graphs.
- To explore neighbours first
- To ~~check~~ check connectivity
- To find minimum steps

Real life applications

- ① shortest path in maps
- ② social networking
- ③ web
- ④ Broadcasting systems
- ⑤ finding minimum Moves in games.

⑤ please compare of "breadth first search" and "depth first search"

feature

BFS

DFS

full form → Breadth first search Depth first Search

Data structure → Queue stack / Recursion

Traversal → level by level Depth wise

memory usage → More less

Shortest path → Yes (Weighted) No guarantee

Backtracking → No Yes

Abhinav
10-3-16

Q1 Implement random funcⁿ to generate marks of 30 students using "random.randint()".

Q2 Implement A-star

Q3 Implement BFS.

Solⁿ 1 import numpy as np
import random

r_data = np.random.randint(1, 101, size=30)
marks = list(r_data)
print(marks)

output = [np.int32(34), np.int32(59), ..., np.int32(83)]

Solⁿ 2 # A* code

import networkx as nx
graph = nx.Graph()

while True:

choice = int(input("want to insert node
and enter edges? 1 for yes & 2 for no"))

if choice == 1:

node1 = input("enter node 1:").strip().upper()

node2 = input("enter node 2:").strip().upper()

weight = int(input("enter weight b/w
node 1 and node 2:"))

graph.add_edge(node1, node2, weight=weight)

```

else:
    break
graph.adj.items()
for node, neighbours in graph.adj.items():
    print(node, neighbours)

```

```

# output => A{'B': {'weight': 5}, 'C': {'weight': 3}}
           B{'A': {'weight': 5}, 'C': {'weight': 4}}
           C{'A': {'weight': 3}, 'B': {'weight': 4}}

```

```

start = input("enter start node: ").strip().upper()
goal = input("enter goal node: ").strip().upper()

```

```

# output => Enter start node: a
           Enter goal node: c
graph.nodes()

```

```

# output => Node view({'A', 'B', 'C'})

```

```

heuristic_values = {}

```

```

heuristic_values[goal] = 0

```

```

for node in graph.nodes():

```

```

    if node != goal:

```

```

        heuristic_val = int(input("enter heuristic value
                                   from node {node} to goal node
                                   {goal}"))

```

```

        heuristic_values[node] = heuristic_val

```

```

heuristic_values

```


output => {'C': 0, 'A': 2, 'B': 3}

```
def get_heuristic(u, v):  
    return heuristic_values[u]
```

```
path = num. astar_path(graph, start, goal,
```

```
    heuristic = get_heuristic,
```

```
    weight = 'weight')"
```

output => 'A -> C'

```
total_cost = num. astar_path_length(graph, start,
```

```
    goal, heuristic = get_heuristic,
```

```
    weight = 'weight')
```

```
print(f" {total_cost}\n")
```

output => 3

Solⁿ 3 => # BFS - code

```
d = {}
```

```
n = int(input("Enter total weight: "))
```

```
for i in range(n):
```

```
    u = input(f"Enter node {i}: ")
```

```
    d[u] = []
```

```
n = int(input("Enter num of neighbours {i}: "))
```

```
for i in range(n):
```

```
    v = input()
```

```
print(d)
```

```
d[u].append(v)
```

```
from collections import deque
def bfs(graph, start_node):
    vis = set()
    queue = deque([start_node])
    vis.add(start_node)
    while queue:
        node = queue.popleft()
        print(node, end = " ")
        for neigh in graph[node]:
            if neigh not in vis:
                vis.add(neigh)
                queue.append(neigh)
```



- I create two datasets one with features area, length, breadth, rooms and target as price. second with features if above 2BHK $\rightarrow$ big apartment
below 2BHK $\rightarrow$ small apartment

Q) Create a dataset to perform logistic regression code

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score, precision_score,  
recall_score, f1_score, confusion_matrix, classification_report
```

```
from sklearn.linear_model import LogisticRegression
```

```
data = {
```

```
    "Study_hours": [1, 2, 3, 4, 5, 6, 7, 8],
```

```
    "Result": [0, 0, 0, 0, 1, 1, 1, 1]
```

```
}
```

```
df = pd.DataFrame(data)
```

df

	Studyhours	output Result
0	1	0
1	2	0
2	3	0
3	4	0
4	5	1
5	6	1
6	7	1
7	8	1

```
x = df[["Study hours"]]
```

```
y = df[["Result"]]
```

x, y

	Study hours.	output Result
0	1	0
1	2	0
2	3	0
3	4	0
4	5	1
5	6	1
6	7	1
7	8	1


```
n_train, n_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2,
    RandomState=42)
```

```
logreg = LogisticRegression()
```

```
logreg.fit(n_train, y_train)
```

```
y_pred = logreg.predict(n_test)
```

```
y_pred
```

OUTPUT

```
array([[0, 1]])
```

```
accuracy_score(y_test, y_pred)
```

```
output: 1.0
```

```
precision_score(y_test, y_pred)
```

```
output: 1.0
```

```
recall_score(y_test, y_pred)
```

```
output: 1.0
```

```
f1_score(y_test, y_pred)
```

```
output: 1.0
```

```
print(confusion_matrix(y_test, y_pred))
```

```
output [[1 0]
        [0 1]]
```

```
print(classification_report(y_test, y_pred))
```

⇒ for KNN → And SUM

import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error,

mean_absolute_error, r2_score, accuracy_score, precision-

score, recall_score, F1_score, confusion_matrix,

classification_report

from sklearn.preprocessing import LabelEncoder

→ data = {

"P-id" = np.random.randint(1, 5000, 5000),

"disease" = np.random.choice(["Yes", "No"], 5000)

}

→ df = ~~pd.DataFrame~~ (data)

df

→ df.duplicated().sum()

⇒ #, Output 1086

n = df[["P-id"]]

y = df[["disease"]]

→ n.y

→ le = LabelEncoder()

df["disease_lb"] = le.fit_transform(df["disease"])


```
* n_train, n_test, y_train, y_test = train_test_split(
    n, df["disease"],
    test_size=0.2, random_state=42)
```

```
* knn = KNeighborsClassifier(n_neighbors=13)
```

```
* knn.fit(n_train, y_train)
```

```
* y_pred = knn.predict(n_test)
```

```
* y_test
```

```
* y_pred
```

```
* accuracy_score(y_test, y_pred)
```

```
* precision_score(y_test, y_pred)
```

```
* recall_score(y_test, y_pred)
```

```
* f1_score(y_test, y_pred)
```

```
* print(confusion_matrix(y_test, y_pred))
```

```
* data = {
```

```
    "p-id": np.random.randint(1, 5000, 5000),
```

```
    "age": np.random.randint(15, 80, 5000),
```

```
    "weight": np.random.randint(35, 80, 5000),
```

```
    "disease": np.random.randint(["BP", "Diabetes",
```

```
    ],
```

```
    ["Cancer", "Asthma",
```

```
    "Jaundice"], 5000)
```

df = pd.DataFrame(data)

df

* n = df[["p-id"]]

y = df[["disease"]]

Q perform KMean code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
ny = make_blobs(n_samples=200, centers=3,
                random_state=12)
```

```
df = pd.DataFrame(ny, columns = ['Feature1', 'Feature2'])
ss = StandardScaler()
```

```
x_scaled = ss.fit_transform()
```

```
for i in range(1,8):
```

```
    km = KMeans(n_clusters=i, random_state=12)
```

```
    km.fit(x_scaled)
```

```
    wcss.append(km.inertia_)
```

```
print('\n wcss values for different k's')
```

```
for i in range(len(wcss)):
```

```
    print('k = {i+1} → wcss = {wcss[i]}')
```

```
plt.plot(range(1,8), wcss, marker='o')
```

```
plt.title('elbow method')
```

```
plt.xlabel('k')
```

```
plt.ylabel('wcss')
```

```
plt.show()
```

```
km = KMeans(n_clusters=3, random_state=12)
```

```
y_km = km.fit_predict(x_scaled)
```

```
print('\n cluster centers')
```

```
print(km.cluster_centers_)
```

```
print('\n cluster labels (first 10 points):')  
print(g_km[:10])
```

output

wcss values for different K :

$$K=1 \rightarrow wcss = 400.23$$

$$K=2 \rightarrow " = 210.45$$

$$K=3 \rightarrow " = 85.67$$

$$K=4 \rightarrow " = 70.1^2$$

$$K=5 \rightarrow " = 60.34$$

$$K=6 \rightarrow " = 52.89$$

$$K=7 \rightarrow " = 48.21$$

cluster centres =

$$\begin{bmatrix} -1.2 & 1.1 \\ 0.9 & -0.8 \\ 1.5 & 1.3 \end{bmatrix}$$

Q perform a linear regression on the dataset

```
import pandas as pd.
```

```
import numpy as np.
```

```
import matplotlib.pyplot as plt
```

```
data = {
```

```
    'name' = ['std.1', 'std.2', 'std.3', 'std.4', 'std.5',  
              'std.6', 'std.7']
```

```
    'age' = [19, 28, 17, 32, 23, 18, 22]
```

```
    'roll no' = [1, 2, 3, 4, 5, 6, 7]
```

```
    'marks' = [98, 94, 90, 76, 82, 60, 85]
```

```
df = pd.DataFrame(data)
```

```
df.to_csv('data1.csv')
```

```
df = pd.read_csv('data1.csv')
```

```
plt.scatter(df['roll no'], df['age'])
```

```
plt.xlabel('Roll no')
```

```
plt.ylabel('age')
```

```
plt.title('Roll No. vs Age')
```

```
plt.show()
```

```
x = df['age']
```

```
y = df['roll no']
```

```
x = np.array(x).reshape(-1, 1)
```

```
y = np.array(y).reshape(-1, 1)
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
x_train, x_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=10)
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
y_pred.

```

outputs:-

Linear Regression (①)	
▲ Parameters	
fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

array([[4.61663067], [4.66414687]])

Abhinav
29-04-20